

计算机系统基础 lab0 作业报告

赵婉扬 25800190042

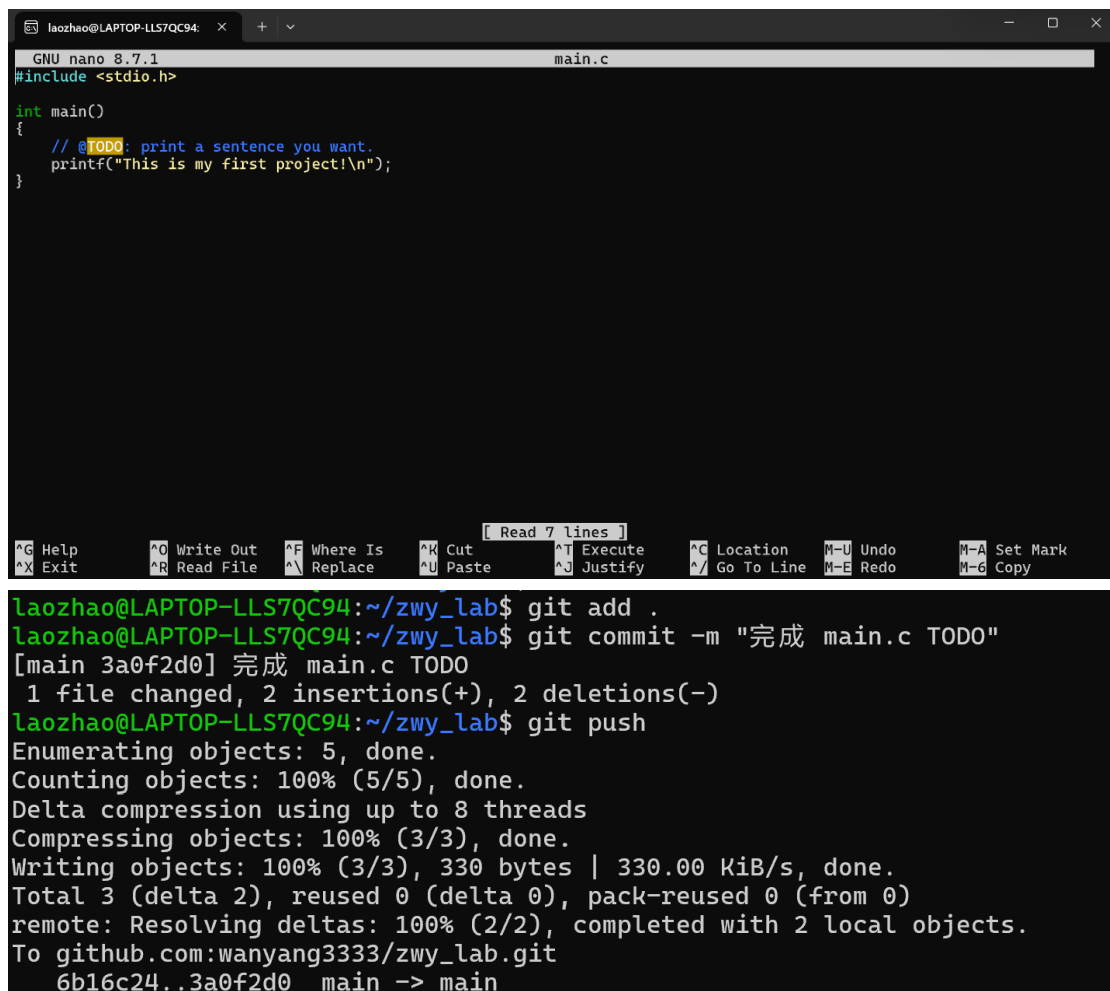
一、

- (1) 打过美赛，但是没有做过“开发”项目。
- (2) 暂存给编辑留了一个缓冲区，使得我们可以分多次编辑，最后检查无误后再有组织地进行提交
- (3) git branch 只列出本地分支，而 git branch -a 列出本地分支+远程跟踪分支，因为这里的 a 是 all 的意思。远程跟踪分支相当于本地 Git 对“远程仓库分支状态”的一份记录，例如：

```
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ git remote -v
origin  git@github.com:wanyang3333/zwy_lab.git (fetch)
origin  git@github.com:wanyang3333/zwy_lab.git (push)
```

二、

- (1) 修改过程：



```
GNU nano 8.7.1 main.c
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("This is my first project!\n");
}

laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ git add .
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ git commit -m "完成 main.c TODO"
[main 3a0f2d0] 完成 main.c TODO
1 file changed, 2 insertions(+), 2 deletions(-)
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 330 bytes | 330.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:wanyang3333/zwy_lab.git
6b16c24..3a0f2d0  main -> main
```

- (2) (不是任务)

三、

- (1)

我阅读了阮一峰的《Commit message 和 Change log 编写指南》。文章介绍了 Angular 的 Commit message 规范，格式为 <type>(<scope>):<subject>, type 分为 feat、fix、docs、style、refactor、test、chore 七类。规范的 commit 便于浏览历史、过滤查找，也能自动生

成 Change log。文章还介绍了 Commitizen、validate-commit-msg、conventional-changelog 等配套工具。

规范的 Commit message 让项目历史清晰可读，多人协作时能快速理解每次改动。这正说明了 Git 的价值：它不只是“保存代码”，更是一套让协作和追溯变得有序的工具。学习 Git，是为了在团队开发中把版本管理这件事做好。

(2)

这两篇分别从提交规范和分支管理两个角度说明了 Git 在团队协作中的价值。Commit message 规范让历史清晰可读、能自动生成 Change log；Gitflow 让不同分支各司其职，避免冲突和版本混乱。可见，学习 Git 的意义是学一套让多人协作有序进行的方法。

四、

先切换到 feature 分支进行修改：

```
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ git switch -c feature
Switched to a new branch 'feature'

GNU nano 8.7.1 main.c *
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("Edited in feature.\n");
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  ^M-U Undo    ^M-A Set Mark
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line ^M-E Redo    ^M-G Copy
```

```
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ nano main.c
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ cat main.c
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("Edited in feature.\n");
}
```

再切换到 main 分支进行修改：

```
laozhao@LAPTOP-LLS7QC94: x + -
GNU nano 8.7.1 main.c *
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("Edited in main.\n");
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  ^U Undo      ^M-A Set Mark
^X Exit      ^R Read File ^A Replace   ^U Paste     ^J Justify   ^_ Go To Line ^-E Redo     ^M-G Copy
```

```
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ git switch main
```

```
M      main.c
```

```
Switched to branch 'main'
```

```
Your branch is up to date with 'origin/main'.
```

```
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ nano main.c
```

```
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ cat main.c
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // @TODO: print a sentence you want.
```

```
    printf("Edited in main.\n");
```

```
}
```

```
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ |
```

```
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ git add .
```

```
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ git commit -m "main 分支修改"
```

```
[main 943c320] main 分支修改
```

```
1 file changed, 1 insertion(+), 1 deletion(-)
```

合并，发现出现冲突：

```
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ git merge feature
```

```
Auto-merging main.c
```

```
CONFLICT (content): Merge conflict in main.c
```

```
Automatic merge failed; fix conflicts and then commit the result.
```

解决冲突：

```
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ cat main.c
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    <<<<<<< HEAD
    printf("Edited in main.\n");
    =====
    printf("Edited in feature.\n");
    >>>>>>> feature
}
```

```
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("Edited in main.\n");
    printf("Edited in feature.\n");
}

[ Read 10 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   W-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/_ Go To Line  W-E Redo      M-6 Copy
```

```
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ nano main.c
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ cat main.c
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.

    printf("Edited in main.\n");

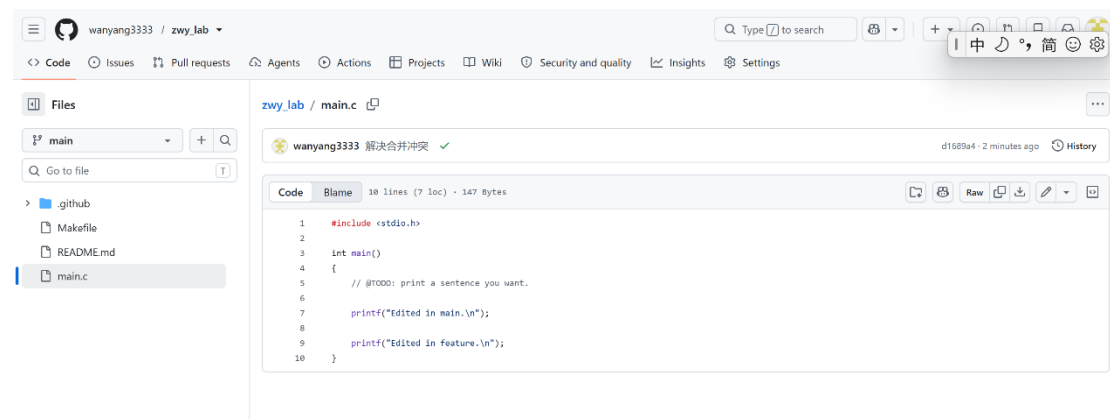
    printf("Edited in feature.\n");
}
```

```

laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ git add .
git commit -m "解决合并冲突"
[main d1689a4] 解决合并冲突
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ git push
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 853 bytes | 853.00 KiB/s, done.
Total 9 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 1 local object.
To github.com:wanyang3333/zwy_lab.git
   3a0f2d0..d1689a4  main -> main
laozhao@LAPTOP-LLS7QC94:~/zwy_lab$ git log --oneline --graph --all
*   d1689a4 (HEAD -> main, origin/main, origin/HEAD) 解决合并冲突
| \
| * df89faa (feature) feature 分支再次修改
* | 943c320 main 分支修改
|/
* 3a0f2d0 完成 main.c TODO
* 6b16c24 Initial commit

```

解决效果：



The screenshot shows the GitHub web interface for the repository 'wanyang3333 / zwy_lab'. The commit history shows a merge of the 'feature' branch into 'main'. The code view shows the final state of main.c, which contains the code from both branches.

```

1  #include <stdio.h>
2
3  int main()
4  {
5      // TODO: print a sentence you want.
6
7      printf("Edited in main.\n");
8
9      printf("Edited in feature.\n");
10 }

```

思考：Git 合并时出现冲突，需要两个分支都修改了同一文件的同一区域，且改成了不同的内容。

五、

(提交 github 中……)